

FINAL REPORT

PROJECT

For the fir filter implementation the number of taps chosen is 16 and the window method to be rectangular window technique low pass filter sampling freq is 10khz and cutoff freq is 2khz. Using python firwin function generated the coefficients needed and it came to be [1.47508243e-17 4.40593836e-02 3.21811414e-02 -3.93325061e-02 -8.18245695e-02 1.47508243e-17 1.90923995e-01 3.53992555e-01 3.53992555e-01 1.90923995e-01 1.47508243e-17 -8.18245695e-02 -3.93325061e-02 3.21811414e-02 4.40593836e-02 1.47508243e-17]. Which is between -1 and .992157 range so decided to use q1.7 fixed point representation.

The uart baudrate used here is 115200 with 10 bit packet consisting of 8 bit data and 1 start bit and 1 stop bit.

The clock frequency of the fpga board is 12mhz.

First created the fir filter implementation and verified it using verilog testbench feeding values to dut through testbench and verified it using the numpy python implementation.

Secondly created the uart modules.

First created the baudrate generator which is basically 16 times the baudrate in order to sample each bits 16 times. Verified it using cocotb testbench.

Similarly created uart rx and tx modules and verified the whole setup of baudgenerator rx and tx modules using a uart echo setup in which rx is connected to tx and verifying if it is correctly showing on the uart terminal.

There were issues in between like some of the characters printing wrong garbage values and all but rectified the errors by printing each value and verifying it and implemented the setup correctly.

Next connected the Fir filter with uart modules and created a top module. Then tested the implementation on the fpga board and successfully implemented it.

Ice40 flow was used. Tools used were yosys nextpnr icepak iceprog. I understood each step and implemented it using scripts. Also created a run.sh file to run everything all at once. The resource utilisation was understood.

also can be implemented using python library apio using apio build and apio upload. Which is same as these but as a shorter command version.

First the values were given as an array of 5 elements and verified the results using the python numpy golden model which was created earlier.

Then at the last stage created a signal and also a noise signal. Added both and then sampled it using the sampling frequency and created an array and sent it through uart and read the output simultaneously.

Verified the functioning of fir filter correctness by plotting the input signal and output signal in both frequency domain and time domain.

It was found that the signals above 3khz were successfully filtered out from the signal.

Also created a golden model and generated random value arrays and send it and captured output and the truncated output and the value of the numpy truncated came out same.

```
(.venv) kevin@kevin:~/Desktop/uart_streamed_fir_filter/verilog/final_code$ python3 goldenmodelverification.py
Sent: [14, 37, 5, 96, 67, 58, 21, 0, 71, 40, 79, 6, 77, 43, 52, 56]
Received : [0, 0, 1, 2, 1, 1, 3, 3, 4, 12, 33, 57, 70, 58, 36, 23]
16
16
numpy model convolution values [ 0  70 241 103 175 324 364 560 1539 4200 7315 8907 7429 4598 2977]
trunc op of convolve implementation [0, 1, 2, 1, 1, 3, 3, 4, 12, 33, 57, 70, 58, 36, 23]
results matched
```

LESSONS LEARNT

Learned about the fir filters the mathematics behind it, choosing window function , convolution and all.

Fixed point representation the overflow handling techniques Quantisation modes why fixed point representation is preferred over floating point in fpgas.

Uart communication implementation taught me about the difference between implementation on real fpga vs just simulation. Things like metastability in pins and all were learned at this stage.

Fir filter implementation taught more about handling the values and how to use quantisation and overflow handling without losing precision. I havent done any optimisation in the fpga in this project. But in the bruno levy implementation of riscv processor i learnt about optimisation and using single hardware for multiple instructions and all.

Shifting from verilog testbenches to cocotb testbench. Verilog testbenches are not flexible. In this case if i need to compare the values with numpy golden model or i need to generate random test cases and all then verilog testbenches are not actually on a upperhand in this case. I used cocotb verification technique. Only used some basic implementation and all. Need to learn uvm methodology and systemverilog testbench after this.

Have studied the fft implementation and signal things using numpy libraries. Previously used matlab for this case scenarios but now comfortable with generating signals finding fft and plotting using matplotlib. Previously only used numpy for array manipulations and machine learning. Didnt have used it for signal processing.

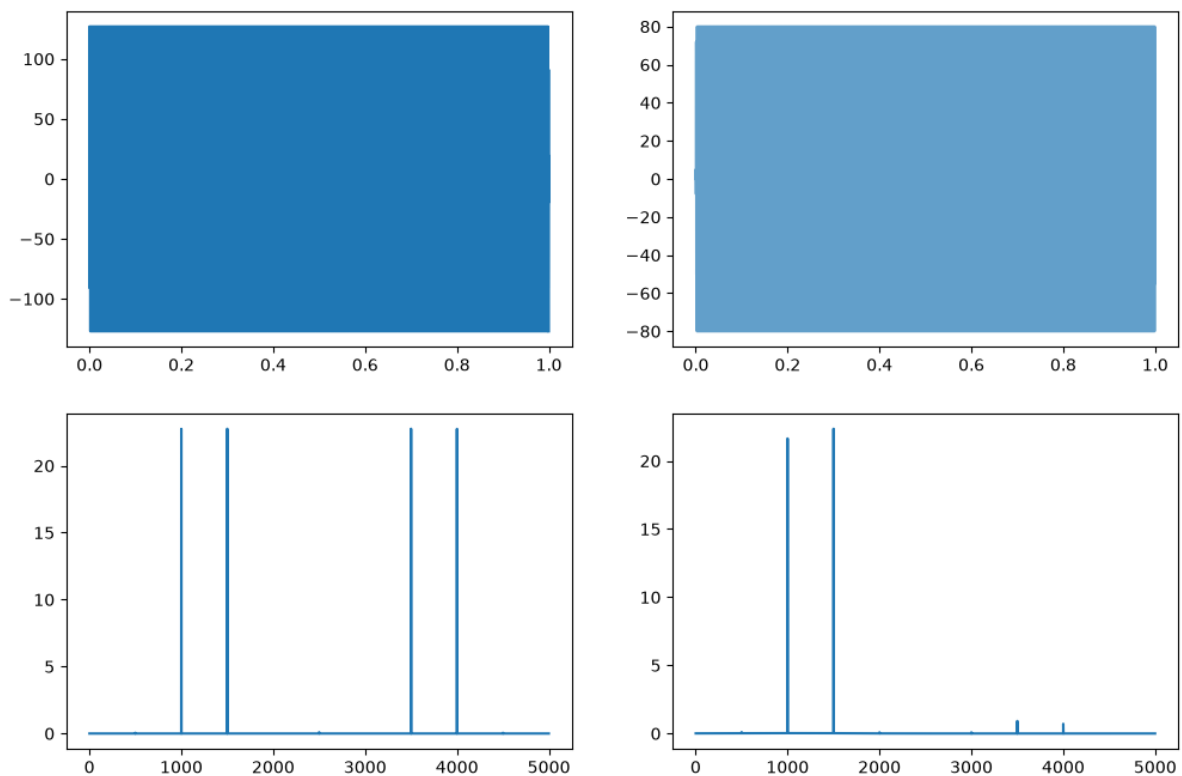
Learnt about the ice40 flow yosys synthesys , using nextpnr viewing the resource utilisation of logic cells and calculating how many percentage is utilised.

The max clock freq after the clock is 16.54 mhz but we are using 12mhz so there is a positive margin so no problem.

```
Info: Max frequency for clock 'clk$SB_IO_IN_$glb_clk': 16.54 MHz (PASS at 12.00 MHz)
```

Info:	ICESTORM_LC:	986/	5280	18%
Info:	ICESTORM_RAM:	0/	30	0%
Info:	SB_IO:	4/	96	4%
Info:	SB_GB:	3/	8	37%
Info:	ICESTORM_PLL:	0/	1	0%
Info:	SB_WARMBOOT:	0/	1	0%
Info:	ICESTORM_DSP:	0/	8	0%
Info:	ICESTORM_HFOSC:	0/	1	0%
Info:	ICESTORM_LFOSC:	0/	1	0%
Info:	SB_I2C:	0/	2	0%
Info:	SB_SPI:	0/	2	0%
Info:	IO_I3C:	0/	2	0%
Info:	SB_LEDDA_IP:	0/	1	0%
Info:	SB_RGBA_DRV:	0/	1	0%
Info:	ICESTORM_SPRAM:	0/	4	0%

Input vs output signal



The signal was sent to uart using python.

Code Repo structure

filter_design_python **flow** **Readme.md** **requirements.txt** **resources**
verilog

start by cloning the repo

https://github.com/kevinroshann/uart_streamed_fir_filter

create a virtual environment

```
python3 -m venv .venv
```

```
source .venv/bin/activate
```

then install the requirements file using pip.

The filter_design_python folder contains the coefficients generation and and numpy golden model code.

The flow folder contains the full ice40 flow of blinking led on fpga board

the Resources folder contain the reports and images

the verilog Folder contains 4 sections

final_code fir_filter top_module uart

the fir_filter contains the fir filter code

the uart contain the uart implementation code

the top_module contains the full code.

The final_code contains the full organised file structure of complete code.

Go to final_code and then run the script to program the board. ./run.sh

then go to the signalsending folder which contain the script to send the wave and run the serialsend.py file to send the signal to the uart.

Or using a script

./full_run.sh

goldenmodel in /verilog/final_code/

github: https://github.com/kevinroshann/uart_streamed_fir_filter